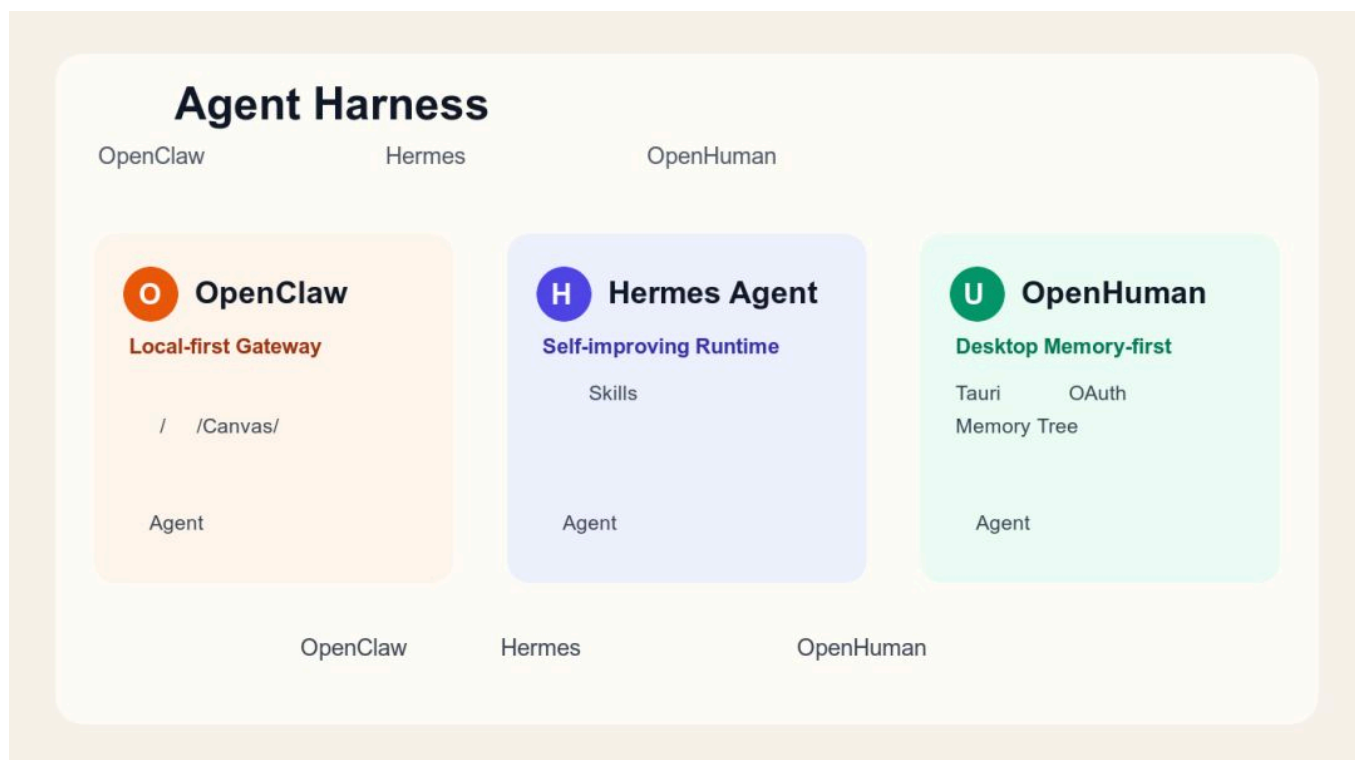


三个 Agent Harness 框架对比：OpenClaw、Hermes Agent、OpenHuman 到底差在哪？

前言



三类 Agent Harness 的不同重心

Agent 框架越来越多，但很多项目表面都在说“让 AI 调工具、执行任务、拥有记忆”，真正落到架构上，差异其实非常大。

这次选三个方向明显不同的项目放在一起看：

- [openclaw/openclaw\[1\]](#)
- [NousResearch/hermes-agent\[2\]](#)
- [tinyhumansai/openhuman\[3\]](#)

它们都可以被理解为某种 AI Agent harness，但重点完全不同：

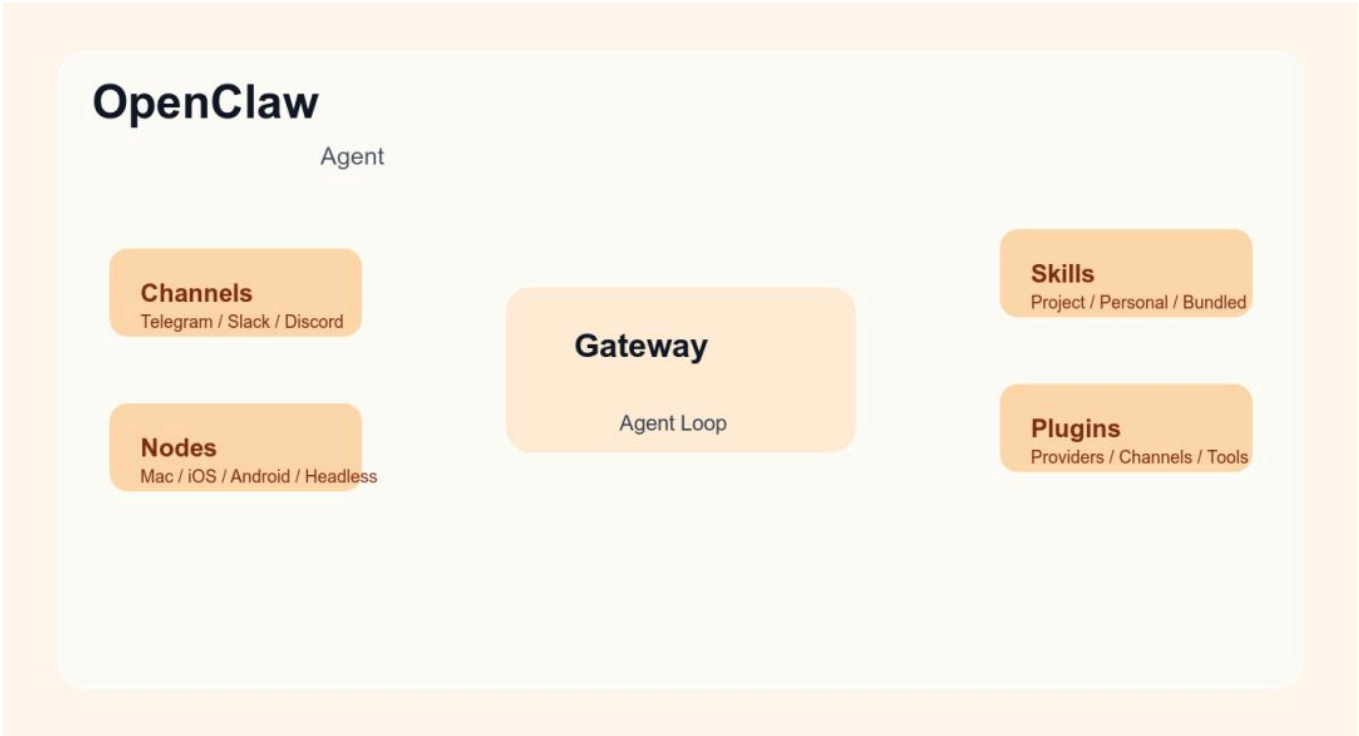
- **OpenClaw** 解决的是入口与控制平面问题：怎么让 Agent 常驻在用户已有设备和聊天渠道里。
- **Hermes Agent** 解决的是自我演化运行时问题：怎么让 Agent 从经验中沉淀 skills、记住用户、跨环境执行。

• **OpenHuman** 解决的是个人上下文和产品体验问题：怎么让 Agent 快速接入个人数据，并通过桌面 UI 变得可用。

一句话概括：

项目	核心定位	架构关键词
OpenClaw	跑在自己设备上的个人 AI 助理，通过各种聊天/设备入口驱动任务	Local-first Gateway、Channels、Nodes、Skills、Plugins
Hermes Agent	会积累记忆、生成/改进 skills、跨平台运行的自改进 Agent	AI Agent Loop、Tool Registry、SQLite/FTS5、Skills Learning、Gateway
OpenHuman	UI 优先、连接个人账号数据并构建长期记忆树的桌面 AI 助理	Tauri/Rust Core、Memory Tree、OAuth Integrations、MCP/Socket.io

OpenClaw：本地优先的个人助理网关



OpenClaw 架构重点

OpenClaw 的核心不是“做一个聊天机器人”，而是做一个**本地优先的个人 AI 控制平面**。

它的 README 里明确强调：OpenClaw 是运行在用户自己设备上的 personal AI assistant。用户可以从 WhatsApp、Telegram、Slack、Discord、Signal、iMessage、WeChat、QQ 等渠道和它交互。

也就是说，它不是只解决“模型如何调用工具”，而是在解决一个更贴近日常的问题：

AI 助理应该住在哪里？用户应该从哪里唤起它？它如何连接桌面、手机、聊天软件和工具？

架构模式：Gateway-centric

OpenClaw 的架构非常典型：以 Gateway 为中心。

Gateway 是长期运行的控制平面，负责：

- 消息渠道接入
- 会话管理
- 工具执行
- 事件分发
- 节点连接
- 权限与路由

CLI、macOS App、Web UI、自动化任务可以通过 WebSocket 连接到 Gateway。macOS、iOS、Android、headless nodes 也可以接入，并声明自己的能力，例如 canvas、camera、screen、location 等。

Agent loop 大致经过：

1intake：接收用户输入

2context assembly：组装上下文

3model inference：模型推理

4tool execution：工具执行

5streaming：流式返回

6persistence：持久化记录

每个 session 会串行执行，避免工具调用和 transcript 写入互相打架。

扩展方式

OpenClaw 的扩展主要来自两层：

- **Skills**: 采用 AgentSkills 兼容目录，支持 workspace、project、personal、managed、bundled 等多级优先级。

- **Plugins**: 用于 provider、channel、tool lifecycle hook 等更底层的扩展。

所以 OpenClaw 更像一个“AI 助理操作系统入口层”。它的价值不只是模型能力，而是把用户已有入口整合起来。

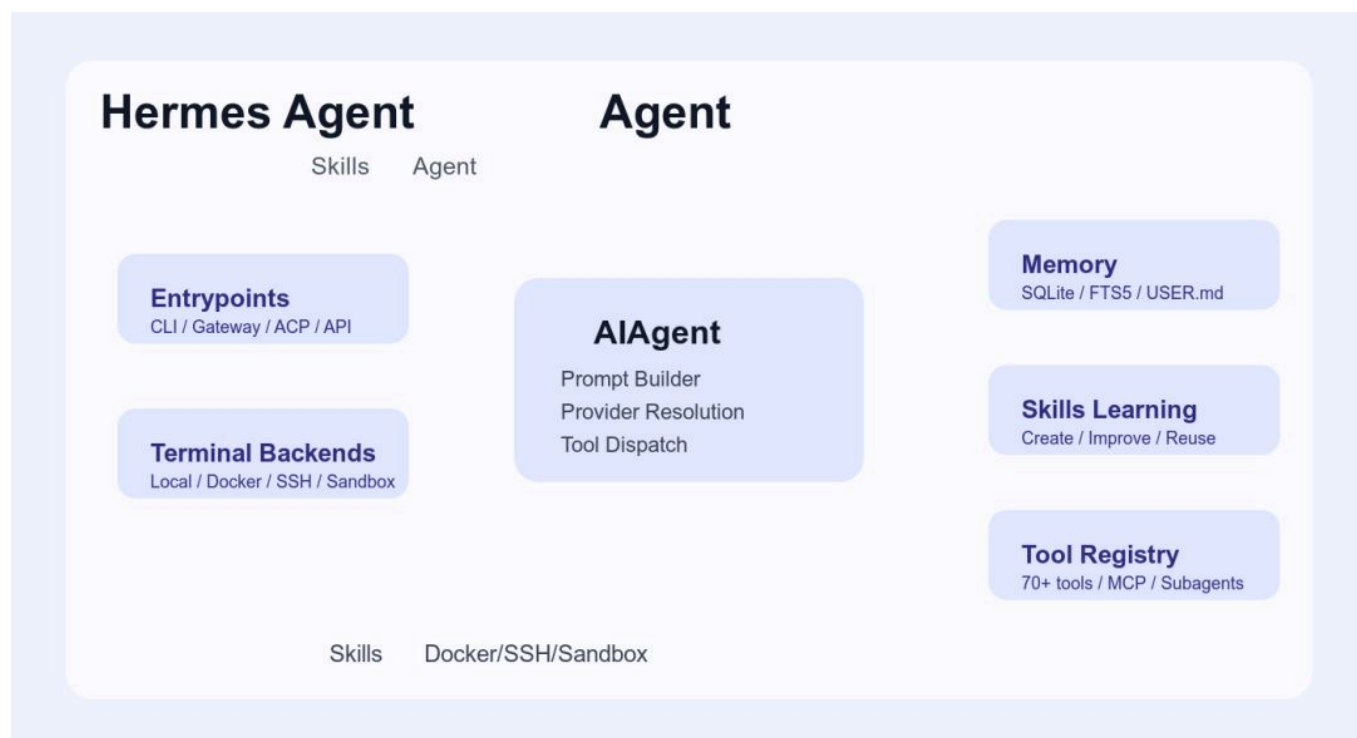
适合什么场景？

OpenClaw 适合：

- 希望 AI 助理常驻在本机或个人服务器上
- 需要从多个聊天渠道发任务、收结果
- 需要连接桌面、手机节点、Canvas、语音等本地能力
- 愿意自己管理模型、配置、权限和安全边界

如果目标是“让 AI 助理住进用户已有设备和聊天入口”，OpenClaw 是三个项目里最贴近这个方向的。

Hermes Agent：会自我改进的 Agent 运行时



Hermes Agent 架构重点

Hermes Agent 的关键词是 **self-improving**。

它更像一个通用 Agent runtime：既能运行任务，也能积累记忆、生成 skills、改进 skills、搜索历史会话，并跨 session 建立对用户的长期理解。

和 OpenClaw 相比，它不那么强调“入口住在哪里”，而更强调：

Agent 如何从一次次执行中学到东西，并把经验沉淀为可复用能力？

架构模式：Agent-loop-centric runtime

Hermes 的中心是 **AI Agent**。

它负责：

- prompt builder
- provider resolution
- tool dispatch
- memory injection
- skill invocation
- session persistence

Hermes 支持多种入口：

- CLI
- Gateway
- ACP
- Batch Runner
- API Server
- Python Library

Provider 层支持多种 API 模式，例如 chat completions、Codex responses、Anthropic messages。

工具系统也很重：Tool Registry 集中注册 70+ tools 和约 28 个 toolsets。终端 backend 支持 local、Docker、SSH、Singularity、Modal、Daytona、Vercel Sandbox 等执行环境。

记忆与自我改进

Hermes 的记忆机制很突出。

它使用 SQLite + FTS5 做 session storage 和历史检索，同时用 **MEMORY.md**、**USER.md** 这类 curated memory 注入 system prompt。

更关键的是，它强调 skills learning：

- 从经验中创建 skill

- 在使用中改进 skill
- 用历史会话检索辅助当前任务
- 通过 cron 和 gateway 让任务长期运行

这让 Hermes 的定位更接近“会成长的 Agent harness”。

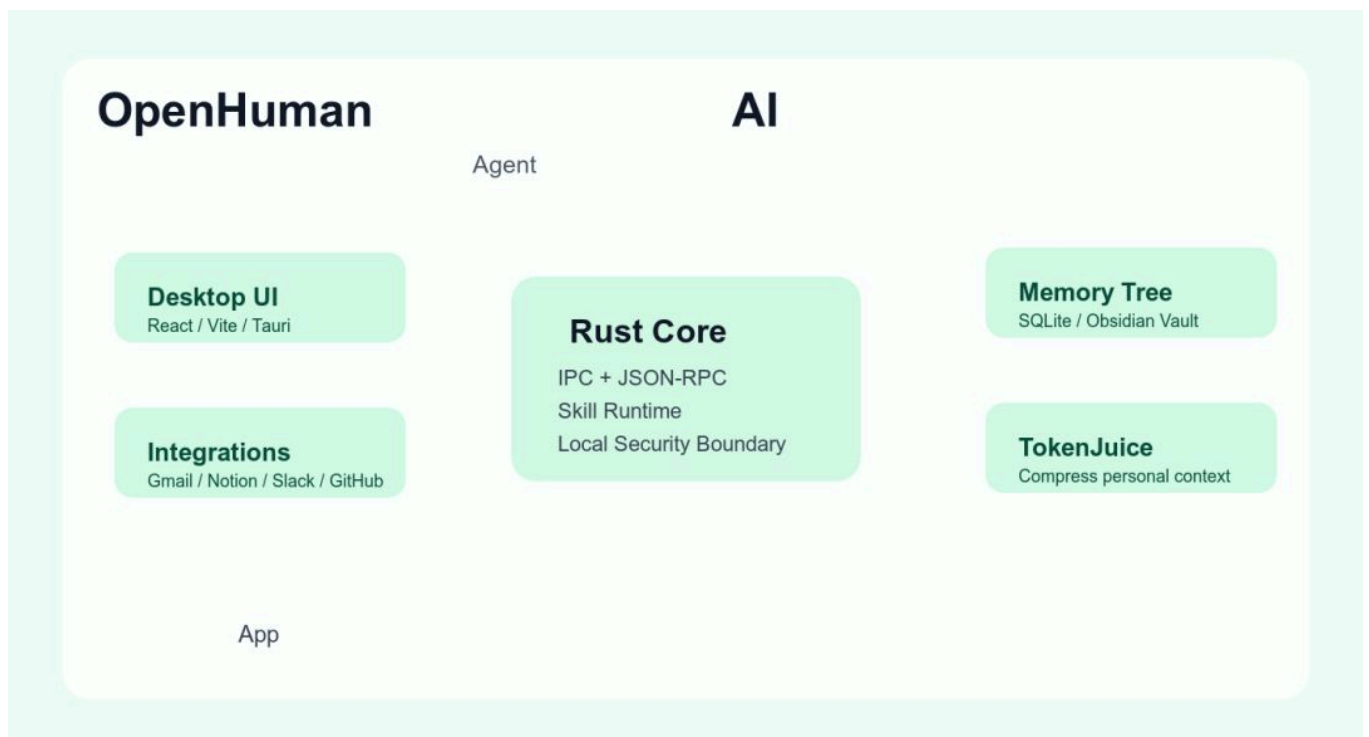
适合什么场景？

Hermes Agent 适合：

- 需要长期自我改进的个人或团队 Agent
- 希望复杂流程沉淀成 skills
- 需要在云 VM、Docker、SSH、serverless sandbox 等环境运行
- 重视 provider/model 灵活切换和工具运行环境可移植性
- 想用 Python runtime 管理工具、记忆、子代理、cron 和 gateway

如果目标是“让 Agent 从经验里变强”，Hermes 是三个项目里最强调这个方向的。

OpenHuman：个人数据驱动的桌面 AI 助理



OpenHuman 架构重点

OpenHuman 的方向更产品化，也更 UI-first。

它强调的是：通过账号集成和自动拉取，让 AI 助理尽快理解用户的个人上下文。

这和前两个项目不太一样。OpenClaw 更像入口网关，Hermes 更像自我演化运行时，OpenHuman 则更像一个面向普通用户的桌面 AI 产品底座。

它想解决的问题是：

Agent 如何快速接入用户的真实工作数据，并把这些数据变成长期记忆？

架构模式：desktop-memory-first

OpenHuman 当前以桌面端为主。

架构上：

- 前端使用 React/Vite
- 桌面壳使用 Tauri
- 核心逻辑在 Rust `openhuman-core`
- WebView 负责 UI
- Rust core 负责 RPC、skills、memory、socket 等能力
- 前后端通过 Tauri IPC 和 HTTP JSON-RPC 通信

它的重点不是只做一个 agent loop，而是围绕个人数据构建一套同步、压缩、记忆和调用系统。

集成与记忆

OpenHuman 提供大量第三方集成，例如 Gmail、Notion、GitHub、Slack、Stripe、Calendar、Drive、Linear、Jira 等，通过 OAuth 接入。

数据同步后，会进入 Memory Tree：

- 把不同来源的数据规范化为 Markdown chunks
- 存入 SQLite
- 写入 Obsidian-compatible vault
- 使用 TokenJuice 压缩工具结果、抓取内容、邮件正文等上下文

这意味着 OpenHuman 的核心竞争力不是“工具特别多”，而是“用户上下文来得快”。

安全边界

OpenHuman 也很强调本地安全：

- 本地存储
- OS keychain

- AES-256-GCM
- Argon2id
- prompt injection guard

这对一个连接大量个人账号的桌面 AI 助理来说非常关键。

适合什么场景？

OpenHuman 适合：

- 希望桌面 UI 开箱即用
- 希望连接 Gmail、Slack、Notion、Drive、Calendar 等个人工作数据
- 希望本地形成 Obsidian/Memory Tree 知识库
- 不想为每个能力单独配置 API key、插件和工具链
- 接受项目仍处于 early beta

如果目标是“让 Agent 先理解用户，再帮用户做事”，OpenHuman 的方向最直接。

横向对比

维度	OpenClaw	Hermes Agent	OpenHuman
主目标	本地常驻个人助理	自改进通用 Agent runtime	UI-first 个人超级助理
核心架构	Gateway 控制平面	AI Agent 统一执行环	Tauri/Rust core + React UI
入口	多聊天渠道、CLI、App、Web、Nodes	CLI、Messaging Gateway、ACP、API、Batch	桌面 UI、集成数据、消息渠道
工具系统	Gateway tools + plugins + skills	70+ tools、28 toolsets、MCP、terminal backends	MCP tool catalog、skill bridge、native tools

Skills	AgentSkills-compatible, 多级优先级, ClawHub	自我创建/改进 skills, Skills Hub	skill packages + managed Node runtime
记忆	session/workspace/context + skills; 偏运行上下文	MEMORY.md、USER.md、SQLite FTS5 session search	Memory Tree、SQLite、Obsidian vault、auto-fetch
集成取向	通讯渠道、设备节点、Canvas、语音、本地工具	Provider、terminal、browser、messaging、cron、subagents	OAuth SaaS 连接器、个人数据同步
部署形态	本机/个人服务器 Gateway	本机、VPS、Docker、SSH、serverless sandbox	桌面 App 为主
安全重点	DM pairing、allowlist、sandbox、gateway auth	command approval、DM pairing、container isolation、profile isolation	本地加密、OS keychain、prompt injection guard
最适合	AI 助理住在用户设备和聊天入口里	Agent 持续学习并把流程沉淀成技能	快速接入个人数据并形成长期上下文

怎么选？

如果目标是让 AI 常驻在多个聊天渠道里，能控制本机、手机、Canvas、语音入口，优先看 OpenClaw。

如果目标是构建一个能长期学习、搜索历史、生成技能、跨环境执行复杂任务的 Agent harness，优先看 Hermes Agent。

如果目标是给普通用户一个桌面 AI 助理，快速连接个人账号、自动同步上下文、形成长期记忆库，优先看 OpenHuman。

更抽象一点：

- OpenClaw 解决的是**入口和控制平面**问题。
- Hermes Agent 解决的是**Agent 自我演化和执行运行时**问题。
- OpenHuman 解决的是**个人上下文获取和产品化体验**问题。

这三个项目放在一起看，刚好对应 Agent 产品化的三条路径：

1先把入口打通，让 Agent 能被随时唤起。

2再把经验沉淀下来，让 Agent 越用越强。

3最后把个人数据接进来，让 Agent 真正理解用户。

未来的 Agent 框架，很可能不会只走其中一条路。真正成熟的系统，最终会同时拥有入口、记忆、工具、数据和安全边界。但在当前阶段，先看清每个框架的重心，反而比追一个“大而全”的答案更重要。

参考来源：

- OpenClaw GitHub[4]
- Hermes Agent GitHub[5]
- OpenHuman GitHub[6]

参考链接

[1] openclaw/openclaw: <https://github.com/openclaw/openclaw>

[2] NousResearch/hermes-agent: <https://github.com/NousResearch/hermes-agent>

[3] tinyhumansai/openhuman: <https://github.com/tinyhumansai/openhuman>

[4] OpenClaw GitHub: <https://github.com/openclaw/openclaw>

[5] Hermes Agent GitHub: <https://github.com/NousResearch/hermes-agent>

[6] OpenHuman GitHub: <https://github.com/tinyhumansai/openhuman>

[7] OpenClaw: <https://github.com/openclaw/openclaw>

[8] Hermes Agent: <https://github.com/NousResearch/hermes-agent>

[9] OpenHuman: <https://github.com/tinyhumansai/openhuman>